

4.

ANALISIS ALGORITMA REKURSIF

4.1. Relasi Rekurens

Relasi rekurens atau *recurrence relation* adalah persamaan atau ketidaksamaan yang mendefinisikan suatu fungsi (seperti waktu berjalan $T(n)$ atau jumlah operasi $M(n)$) atau urutan secara **implisit** sebagai fungsi dari nilainya pada titik lain (input yang lebih kecil). Relasi rekurens merupakan analisis algoritma yang menggunakan teknik rekursif.

Menentukan solusi dari suatu urutan, relasi rekurens memerlukan 2 komponen, yaitu:

1. Langkah rekuren
Merupakan persamaan yang mendefinisikan $T(n)$ berdasarkan $T(n - 1)$ atau $T(n / b)$, dan seterusnya.
2. Kondisi awal (*Base Case*)
Merupakan nilai yang dibutuhkan agar urutan dimulai, atau **kondisi penghenti** bagi fungsi rekursif. Sehingga memastikan bahwa rekursi akan “berhenti” ketika ukuran subproblem cukup kecil. Maka dari itu, *base case* sangat penting dalam relasi rekurens karena jika tidak ada *base case* akan ada tak hingga urutan yang memenuhi relasi rekurens tersebut.

Analogi sederhana relasi rekurens merupakan suatu resep yang sudah spesifik urutannya. Langkah rekuren merupakan urutan membuat makanan “n”, dimana harus melakukan urutan $n - 1$ dan $n - 2$. Sedangkan *base case* adalah bahan dasarnya (misalnya $M(0)$ atau $F(0)$) yang membuat resep tersebut selesai atau berhenti.

Menetapkan relasi rekurens merupakan *step* yang penting dalam menganalisis efisiensi waktu algoritma rekursif, caranya adalah:

1. Menentukan ukuran input (n)
2. Identifikasi operasi dasar algoritma (contohnya perkalian atau pembagian)
3. Menentukan relasi rekurens dan *base case* untuk jumlah eksekusi operasi dasar
4. Selesaikan relasi rekuren atau tentukan *order of growth* (Θ atau O bounds)

Relasi rekurens dapat diklasifikasikan berdasarkan bentuknya:

1. Homogen linear dengan koefisien konstan
Berbentuk $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$. Contohnya adalah relasi fibonacci $f_n = f_{n-1} + f_{n-2}$
2. Nonhomogen linear dengan koefisien konstan
Termasuk fungsi $F(n)$ yang hanya bergantung pada n di sisi kanan, seperti $H_n = 2H_{n-1} + 1$ ²³. Jika rekurensi adalah $a_n = 3a_{n-1} + 2n$, maka $F(n) = 2n$ ²⁴.
3. Nonlinear
Jika koefisiennya bukan konstanta (bergantung pada n), seperti $B_n = n B_{n-1}$

Beberapa metode utama untuk menyelesaikan relasi rekurens dalam analisis algoritma yaitu:

- Metode substitusi mundur
- Metode persamaan karakteristik
- *Master Method*

4.2. Penyelesaian Relasi Rekurens dengan Substitusi

Metode substitusi mundur merupakan salah satu cara untuk menganalisis dan menyelesaikan relasi rekurens. Metode ini termasuk dalam kategori penyelesaian secara **iteratif**. Inti dari metode ini adalah mencari solusi eksplisit (formula yang hanya bergantung pada n) dengan mensubstitusikan atau menggantikan ekspresi rekursif secara berulang kali hingga muncul pola yang mudah dikenali, dan kemudian menggunakan *base case* untuk mendapatkan hasil akhir. Langkah-langkah penyelesaiannya adalah:

1. **Mulai dari Persamaan Rekurens:** Tuliskan relasi rekurens $T(n)$ dalam bentuk aslinya.
2. **Substitusi Mundur:** Ganti $T(n-1)$ (atau $T(n/b)$, dll.) dengan definisinya, lalu ganti $T(n-2)$, dan seterusnya. Proses ini disebut *work backward*.
3. **Identifikasi Pola:** Setelah melakukan i kali substitusi, rumuskan ekspresi umum $T(n)$ yang melibatkan $T(n-i)$ dan term-term non-rekursif lainnya.
4. **Terapkan Base Case:** Tentukan nilai i yang membuat argumen rekursif mencapai kondisi basis (misalnya $n-i = 0$ atau $n-i = 1$). Gantikan nilai $T(\text{basis})$ ke dalam pola umum untuk mendapatkan solusi eksplisit.

Metode substitusi mundur paling efektif digunakan pada relasi rekurens sederhana (order 1) di mana pola pengurangan input jelas, seperti pengurangan konstan ($n-1$) atau pembagian konstan (n/b).

Contoh penggunaan metode substitusi dalam menghitung faktorial. Relasi rekurens untuk jumlah perkalian $M(n)$ pada algoritma faktorial adalah: $M(n) = M(n-1) + 1$ untuk $n > 0$. Base case: $M(0) = 0$.

Proses Substitusi Mundur:

1. $M(n) = M(n-1) + 1$
2. Ganti $M(n-1)$ dengan $[M(n-2) + 1]$: $M(n) = [M(n-2) + 1] + 1 = M(n-2) + 2$
3. Ganti $M(n-2)$ dengan $[M(n-3) + 1]$: $M(n) = [M(n-3) + 1] + 2 = M(n-3) + 3$

Pola Umum: Setelah i kali substitusi, pola yang muncul adalah: $M(n) = M(n-i) + i$

Penyelesaian Akhir: Karena kondisi inisial diberikan untuk $n=0$, kita substitusikan $i = n$ (sehingga $n-i = 0$): $M(n) = M(n-n) + n = M(0) + n$. Karena $M(0) = 0$ (tidak ada perkalian saat $n=0$), solusi eksplisitnya adalah: $M(n) = n$

Metode ini memiliki keterbatasan dalam analisis algoritma, yaitu:

1. Dependencies terhadap pola
Metode ini bergantung pada kemampuan analisis untuk melihat dan merumuskan pola yang muncul dari substitusi berulang kali. Jika polanya rumit atau tidak teratur, metode ini kurang sesuai.
2. Masalah pembulatan (*floors* dan *ceilings*)
Apabila terdapat fungsi pembulatan seperti $\lceil n/2 \rceil$ atau $\lfloor n/2 \rfloor$, proses substitusi mundur menjadi sulit kecuali jika masalah disederhanakan hanya untuk n yang merupakan pangkat dari b (misalnya $n=2^k$ atau $n=3^k$).
3. Membutuhkan bukti formal
Setelah pola ditemukan, formula eksplisit harus dibuktikan menggunakan induksi matematika. Namun seringkali hanya dilakukan verifikasi.
4. Hanya dapat dilakukan jika relasi rekurens berbentuk homogen linear dengan koefisien konstan.

4.3. Relasi Rekurens Order 2

Relasi rekurens order 2 adalah relasi rekurens dengan kasus di mana $k=2$, yang berarti suku ke- n diekspresikan sebagai kombinasi linear dari dua suku sebelumnya, a_{n-1} dan a_{n-2} .

Bentuk umumnya adalah: $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ di mana c_1 dan c_2 adalah bilangan riil, dan $c_2 \neq 0$.

4.3.1 Relasi Rekurens Homogen Order 2

Relasi rekurens linear homogen derajat k dengan koefisien konstan adalah relasi yang berbentuk $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$, di mana $c_1, c_2, \dots, c_k$ adalah bilangan riil (konstanta) dan $c_k \neq 0$.

Untuk menentukan solusi urutan secara unik, relasi rekurens derajat dua membutuhkan **dua base case** (a_0 dan a_1).

Contoh Kunci: Barisan Fibonacci

Barisan Fibonacci (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...) adalah contoh paling terkenal dari relasi rekurens linear homogen order 2.

Relasi rekurensnya adalah: $F(n) = F(n-1) + F(n-2)$ untuk $n > 1$

Dengan **base case**: $F(0) = 0$, $F(1) = 1$

Dalam bentuk umum $a_n = c_1 a_{n-1} + c_2 a_{n-2}$, Fibonacci memiliki $c_1 = 1$ dan $c_2 = 1$.

Berbeda dengan relasi rekurens order 1 yang dapat diselesaikan dengan metode substitusi mundur, relasi order 2 seperti Fibonacci akan sulit memberikan pola yang dapat dikenali jika diselesaikan dengan substitusi mundur.

Pendekatan sistematis untuk menyelesaikan relasi rekurens homogen linear dengan koefisien konstan adalah dengan mencari solusi berbentuk $a_n = r^n$, di mana r adalah konstanta.

1. Pembentukan Persamaan Karakteristik

Dengan mensubstitusikan $a_n = r^n$ ke dalam relasi rekurens $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ dan membagi dengan r^{n-2} , kita mendapatkan **persamaan karakteristik** dari relasi rekurens tersebut: $r^2 - c_1 r - c_2 = 0$. Solusi dari persamaan ini disebut **akar karakteristik**.

2. Solusi Berdasarkan Akar Karakteristik

Kasus 1: Dua Akar Berbeda ($r_1 \neq r_2$)

Jika persamaan karakteristik $r^2 - c_1r - c_2 = 0$ memiliki dua akar yang berbeda, r_1 dan r_2 , maka barisan $\{a_n\}$ adalah solusi dari relasi rekurens jika dan hanya jika: $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$ untuk $n = 0, 1, 2, \dots$, di mana α_1 dan α_2 adalah konstanta yang ditentukan oleh *base case*.

Contoh (Akar Berbeda): Relasi rekurens $a_n = a_{n-1} + 2a_{n-2}$ dengan $a_0 = 2$ dan $a_1 = 7$.

- Persamaan karakteristiknya adalah $r^2 - r - 2 = 0$.
- Akar-akarnya adalah $r_1 = 2$ dan $r_2 = -1$.
- Solusi umumnya: $a_n = \alpha_1 2^n + \alpha_2 (-1)^n$.
- Dengan menyelesaikan sistem persamaan dari *base case*, didapat $\alpha_1 = 3$ dan $\alpha_2 = -1$.
- Solusi akhir: $a_n = 3 \cdot 2^n - (-1)^n$.

Kasus 2: Satu Akar dengan Multiplisitas Dua (Akar Kembar, $r_1 = r_2 = r_0$)

Jika persamaan karakteristik $r^2 - c_1r - c_2 = 0$ hanya memiliki satu akar, r_0 (akar kembar), maka solusi umumnya tidak lagi cukup hanya dengan $a_n = \alpha_1 r_0^n$. Solusi barisan $\{a_n\}$ adalah solusi dari relasi rekurens jika dan hanya jika: $a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$ untuk $n = 0, 1, 2, \dots$, di mana α_1 dan α_2 adalah konstanta.

Contoh (Akar Kembar): Relasi rekurens $a_n = 6a_{n-1} - 9a_{n-2}$ dengan $a_0 = 1$ dan $a_1 = 6$.

- Persamaan karakteristiknya adalah $r^2 - 6r + 9 = 0$.
- Akar-akarnya adalah $(r - 3)(r - 3) = 0$, sehingga $r_0 = 3$.
- Solusi umumnya: $a_n = \alpha_1 3^n + \alpha_2 n 3^n$.
- Dengan menyelesaikan sistem persamaan dari *base case*, didapat $\alpha_1 = 1$ dan $\alpha_2 = 1$.
- Solusi akhir: $a_n = 3^n + n3^n$.

4.3.2 Relasi Rekurens Nonhomogen

Persamaan ini disebut inhomogen karena sisi kanannya tidak sama dengan nol.

Contoh: Kompleksitas Algoritma Fibonacci Naïve

Saat menganalisis algoritma rekursif untuk menghitung $F(n)$, jumlah penambahan $A(n)$ didefinisikan oleh relasi order 2 yang inhomogen¹⁰²:

$$A(n) = A(n-1) + A(n-2) + 1 \quad \text{untuk } n > 1$$

Dengan *base case* $A(0)=0$ dan $A(1)=0$ ¹⁰³.

Rekurens ini dapat diselesaikan dengan trik khusus yang mereduksinya menjadi rekurens homogen:

1. Tulis ulang persamaan sebagai: $[A(n)+1] - [A(n-1)+1] - [A(n-2)+1] = 0$.
2. Lakukan substitusi $B(n) = A(n) + 1$.
3. Persamaan menjadi homogen: $B(n) - B(n-1) - B(n-2) = 0$.
4. Rekurens untuk $B(n)$ ini sama persis dengan $F(n)$, tetapi dimulai dengan *base case* $B(0) = A(0)+1 = 1$ dan $B(1) = A(1)+1 = 1$.
5. Karena $B(n)$ memenuhi relasi Fibonacci dengan *base case* $B(0)=1$ dan $B(1)=1$, maka $B(n) = F(n+1)$.
6. Oleh karena itu, solusi untuk $A(n)$ adalah: **$A(n) = B(n) - 1 = F(n+1) - 1$**

4.4. Contoh-Contoh Kasus

Relasi rekurens dapat muncul dalam berbagai bentuk tergantung pada struktur rekursi algoritma. Berikut contoh kasus yang merupakan relasi rekurens.

1. Tower of Hanoi

- **Pemodelan:**
Jika operasi dasar adalah memindahkan disk 122, kita perlu menganalisis strategi rekursif¹²³:
 - a. Pindahkan $n-1$ cakram dari tiang 1 ke tiang 2 (membutuhkan $M(n-1)$ gerakan)¹²⁴.
 - b. Pindahkan 1 cakram terbesar (ke- n) dari tiang 1 ke tiang 3 (membutuhkan 1 gerakan)¹²⁵.
 - c. Pindahkan $n-1$ cakram dari tiang 2 ke tiang 3 (membutuhkan $M(n-1)$ gerakan)¹²⁶.
- **Relasi Rekurens:** Jumlah perpindahan $M(n)$ adalah $M(n) = 2M(n-1) + 1$ ¹²⁷.
- **Base Case:** $M(1) = 1$.
- **Solusi (dari Bagian 2):** $M(n) = 2^n - 1$.
- **Implikasi:** Solusi ini menunjukkan pertumbuhan eksponensial. Jika digunakan 64 cakram, jumlah perpindahan adalah $2^{64} - 1$. Jika 1 perpindahan per detik, waktu yang dibutuhkan setara dengan lebih dari 500 miliar tahun.

2. Barisan Fibonacci (Algoritma Rekursif Naïve)

- **Pemodelan (Populasi Kelinci):**

$F(n)$ adalah jumlah pasangan pada bulan ke- n , yaitu jumlah pasangan bulan lalu $F(n-1)$ ditambah pasangan baru yang lahir dari yang berusia 2 bulan $F(n-2)$. Ini menghasilkan $F(n) = F(n-1) + F(n-2)$.

- **Pemodelan (Analisis Algoritma):** Algoritma rekursif yang naïve untuk menghitung $F(n)$ berdasarkan definisi, menghasilkan relasi untuk jumlah penambahan $A(n)$: $A(n) = A(n-1) + A(n-2) + 1$.
- **Base Case:** $A(0) = 0$ dan $A(1) = 0$.
- **Solusi (dari Bagian 3):** $A(n) = F(n+1) - 1$.
- **Implikasi:** Karena $A(n)$ tumbuh sebanding dengan $F(n+1)$, kompleksitasnya adalah $\Theta(\phi^n)$. Algoritma ini sangat tidak efisien karena menghitung nilai fungsi yang sama berulang kali.